

# EXPERIMENT 3

## Survey Report on Emerging Trends in Low-Code Application Development

### 1. AIM

To study the emerging trends in low-code application development, understand how low-code platforms differ from traditional application development, identify their benefits and challenges, and compare commonly used low-code application development platforms.

### 2. TRADITIONAL APPLICATION DEVELOPMENT VS EMERGING TRENDS IN LOW-CODE APPLICATION DEVELOPMENT

Traditional application development generally requires developers to write application logic, user-interface code, database operations and integration code manually. It provides extensive control and customization, but development can take considerable time and requires skilled programmers.

Low-code application development uses visual development environments, drag-and-drop components, reusable templates, pre-built connectors and model-driven development. Developers can build applications with much less hand-written code. The current trend is toward combining low-code with cloud services, workflow automation, artificial intelligence, APIs and enterprise integrations.

| Aspect           | Traditional Development                  | Low-Code Development                                        |
|------------------|------------------------------------------|-------------------------------------------------------------|
| Coding           | Large amount of hand-written code.       | Uses visual tools with limited coding.                      |
| Development time | Usually longer for similar applications. | Often faster because of reusable components.                |
| Skills           | Requires stronger programming expertise. | Can involve professional developers and citizen developers. |
| Customization    | Very high control and flexibility.       | High for supported features, but platform-dependent.        |
| UI development   | Built largely through code.              | Visual drag-and-drop interface builders.                    |
| Integration      | Often requires custom integration code.  | Pre-built connectors and APIs are commonly available.       |
| Maintenance      | Codebase must be maintained manually.    | Platform services can simplify updates and maintenance.     |

### Emerging Trends

- **AI-assisted development:** AI is increasingly used to generate application components, workflows, formulas and code from natural-language requirements.
- **Cloud-first development:** Low-code platforms increasingly provide cloud deployment, storage, security and collaboration features.
- **Workflow automation:** Business processes can be automated using visual workflow designers and event-driven actions.

- **API and system integration:** Connectors and APIs help applications communicate with databases, SaaS products and enterprise systems.
- **Citizen development:** Business users can participate in application development under suitable organizational governance.
- **Mobile and responsive applications:** Many platforms support applications that work across desktop and mobile devices.

### 3. BENEFITS AND CHALLENGES IN LOW-CODE

#### Benefits

- **Faster development:** Visual components and reusable templates reduce repetitive coding.
- **Lower development effort:** Developers can focus on application logic instead of creating every interface element from scratch.
- **Rapid prototyping:** Ideas can be converted into working prototypes quickly.
- **Better collaboration:** Visual models can make requirements easier for technical and non-technical stakeholders to understand.
- **Automation:** Workflows, approvals and repetitive business processes can be automated.
- **Integration support:** Connectors and APIs can simplify integration with external services.
- **Scalability options:** Enterprise low-code platforms can support cloud deployment and organizational applications.

#### Challenges

- **Platform dependency:** Applications may become strongly dependent on the selected vendor and its platform.
- **Customization limits:** Highly specialized requirements may still require traditional programming.
- **Security and governance:** Organizations must properly manage access, data protection, integrations and application permissions.
- **Performance concerns:** Poorly designed applications or platform limitations can affect performance for complex workloads.
- **Vendor lock-in:** Moving an application from one platform to another can require significant effort.
- **Skill requirements remain:** Low-code reduces coding effort but does not eliminate the need for software design, testing, security and database knowledge.

## 4. COMPARISON OF EXISTING LOW-CODE PLATFORMS

| Platform             | Main Strength                                             | Typical Use                                 | Key Characteristic                                    |
|----------------------|-----------------------------------------------------------|---------------------------------------------|-------------------------------------------------------|
| Microsoft Power Apps | Business applications and Microsoft ecosystem integration | Forms, internal apps, workflows             | Strong integration with Microsoft services            |
| OutSystems           | Enterprise application development                        | Web and mobile enterprise applications      | Visual development with extensive enterprise features |
| Mendix               | Collaborative application development                     | Business applications and digital solutions | Model-driven and visual development approach          |
| Appian               | Process automation                                        | Business process and workflow applications  | Strong process and workflow capabilities              |
| Salesforce Platform  | CRM-centered application development                      | CRM extensions and business applications    | Strong Salesforce ecosystem integration               |

**Survey Observation:** The platforms differ mainly in their target users, integration ecosystems, application types and enterprise capabilities. The appropriate platform depends on organizational requirements, existing technology infrastructure, security needs, budget and the complexity of the application.

## 5. RESULT

The survey helped in understanding the growing role of low-code application development. Compared with traditional development, low-code platforms can reduce development effort and accelerate application delivery through visual development, reusable components, workflow automation and integrations. At the same time, organizations need to consider platform dependency, customization, security, performance, governance and vendor lock-in before adopting a low-code solution. Thus, low-code development is an important emerging approach for rapidly developing business and enterprise applications while still requiring proper software engineering practices.